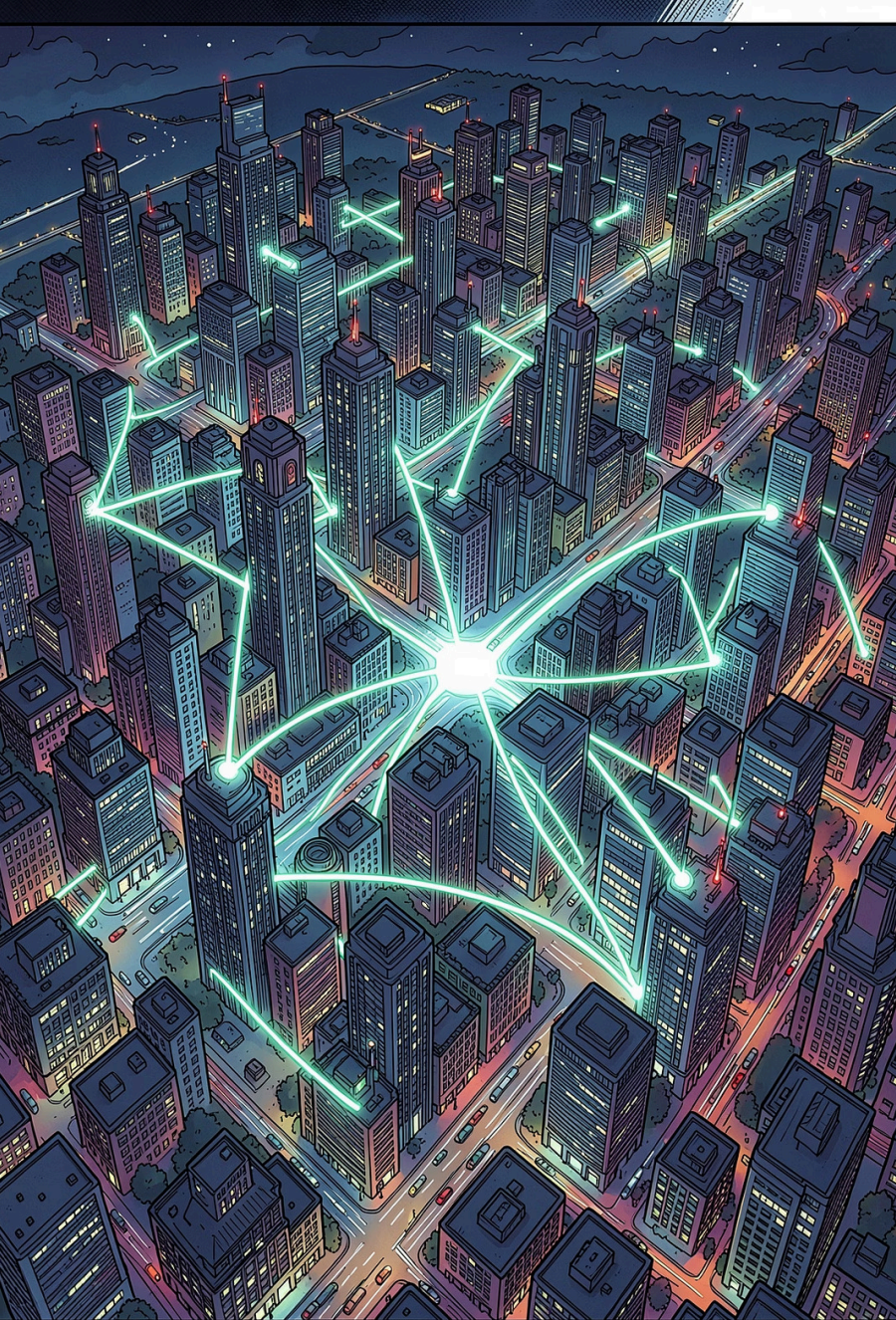


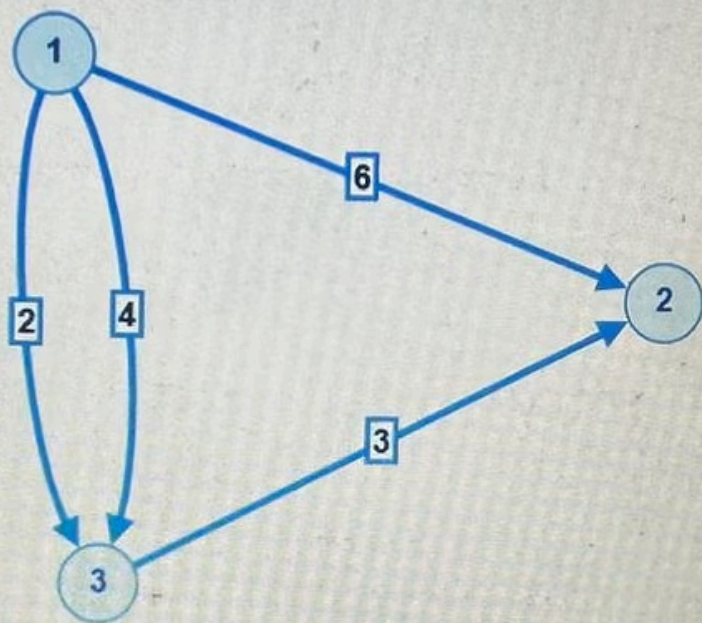


Shortest Routes I

Caminhos Mínimos com Dijkstra

Universidade de Fortaleza (Unifor) · Resolução de Problemas com Grafos

Lucas Peixoto Magalhães · Arthur Alves



Modelagem do Problema

n cidades

m conexões aéreas direcionadas

Custo por voo

Cada aresta tem um peso

Objetivo

Menor distância da cidade **1** para todas as demais

Como Modelamos o Problema



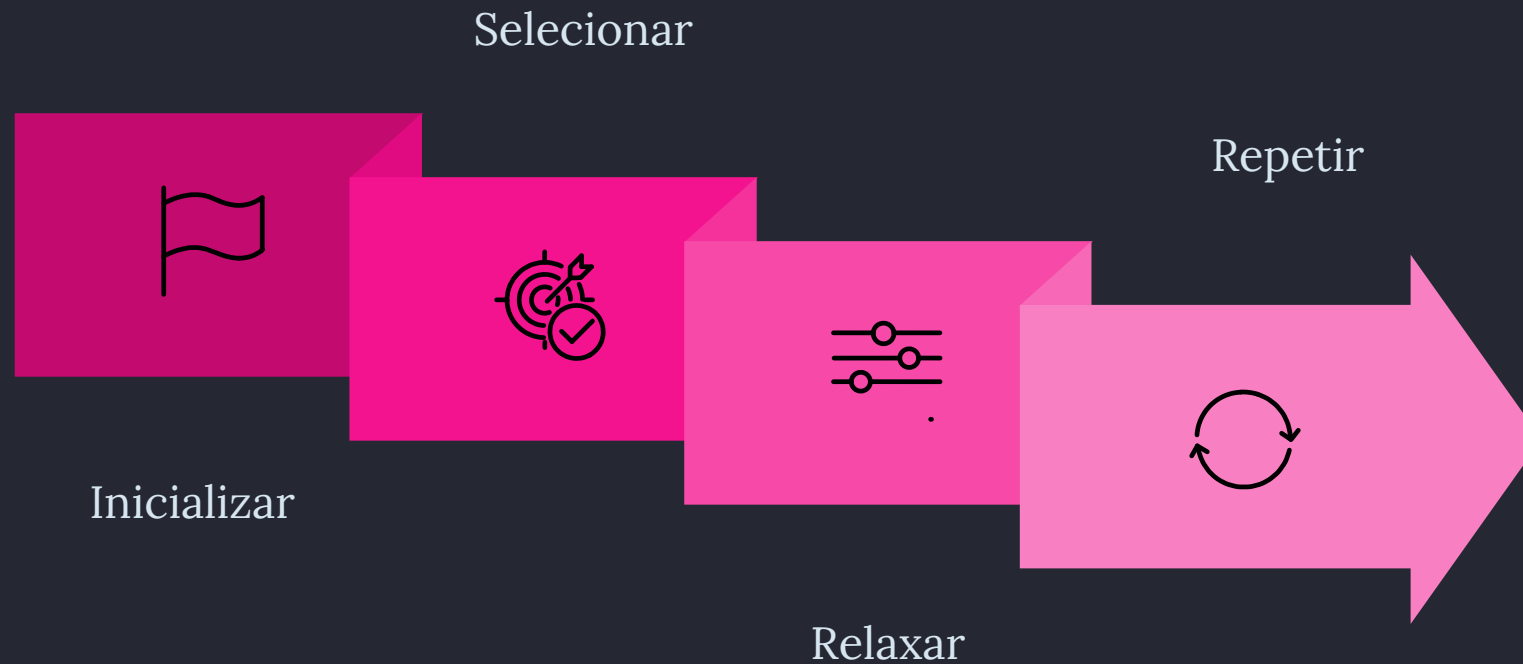
→ Cidade → Vértice

→ Voo → Aresta direcionada

→ Distância → Peso da aresta

→ Origem fixa: cidade 1

Algoritmo de Dijkstra



Pesos positivos
Requisito do algoritmo

Distância inicial ∞
Origem recebe 0

Relaxamento
Atualiza caminhos menores

Otimização da Solução



PriorityQueue (min-heap)

Remove sempre o vértice de menor distância



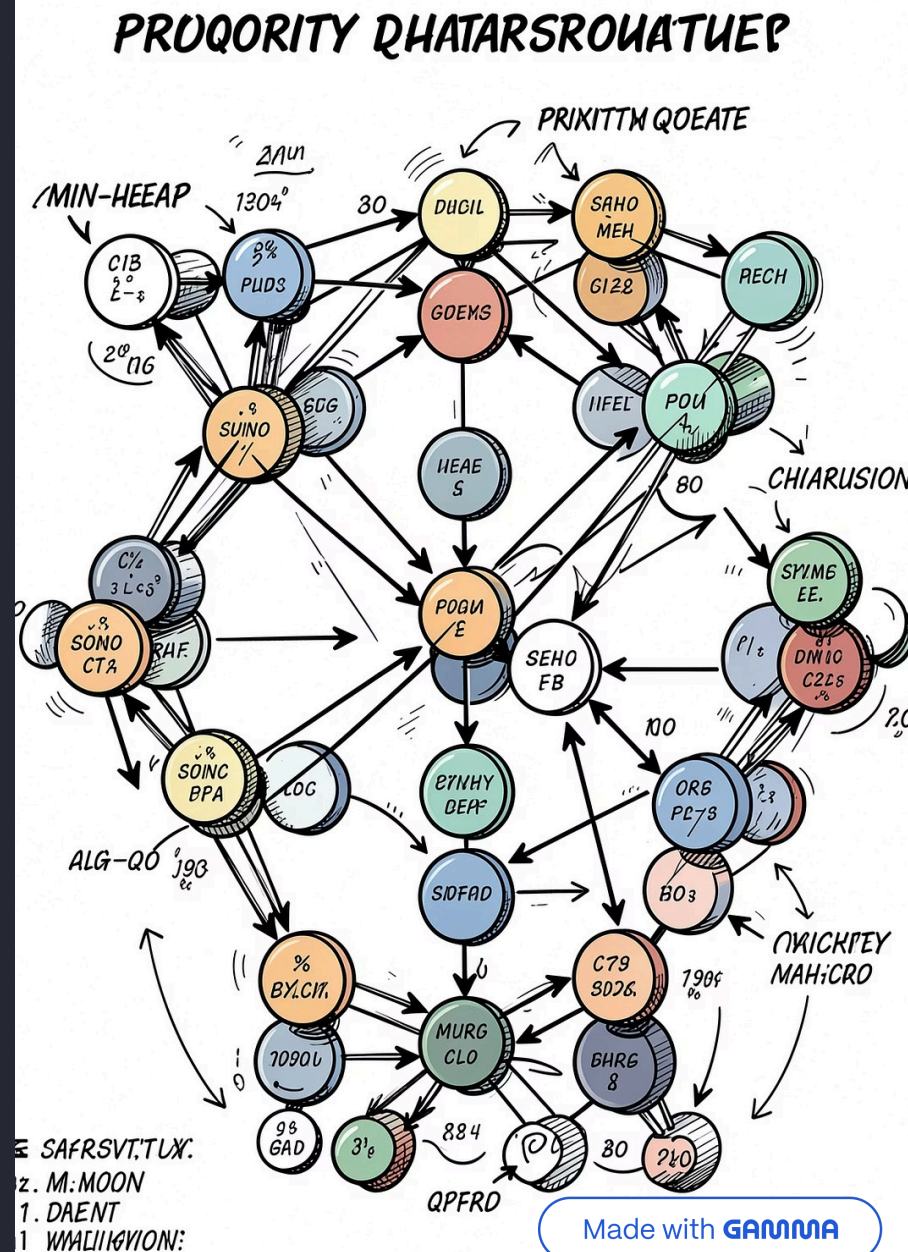
Relaxamento

Atualiza caminhos melhores quando possível



Eficiência

Evita reproprocessamento de vértices



Desempenho da Solução

🕒 Temporal

$O(m \log n)$

📦 Espacial

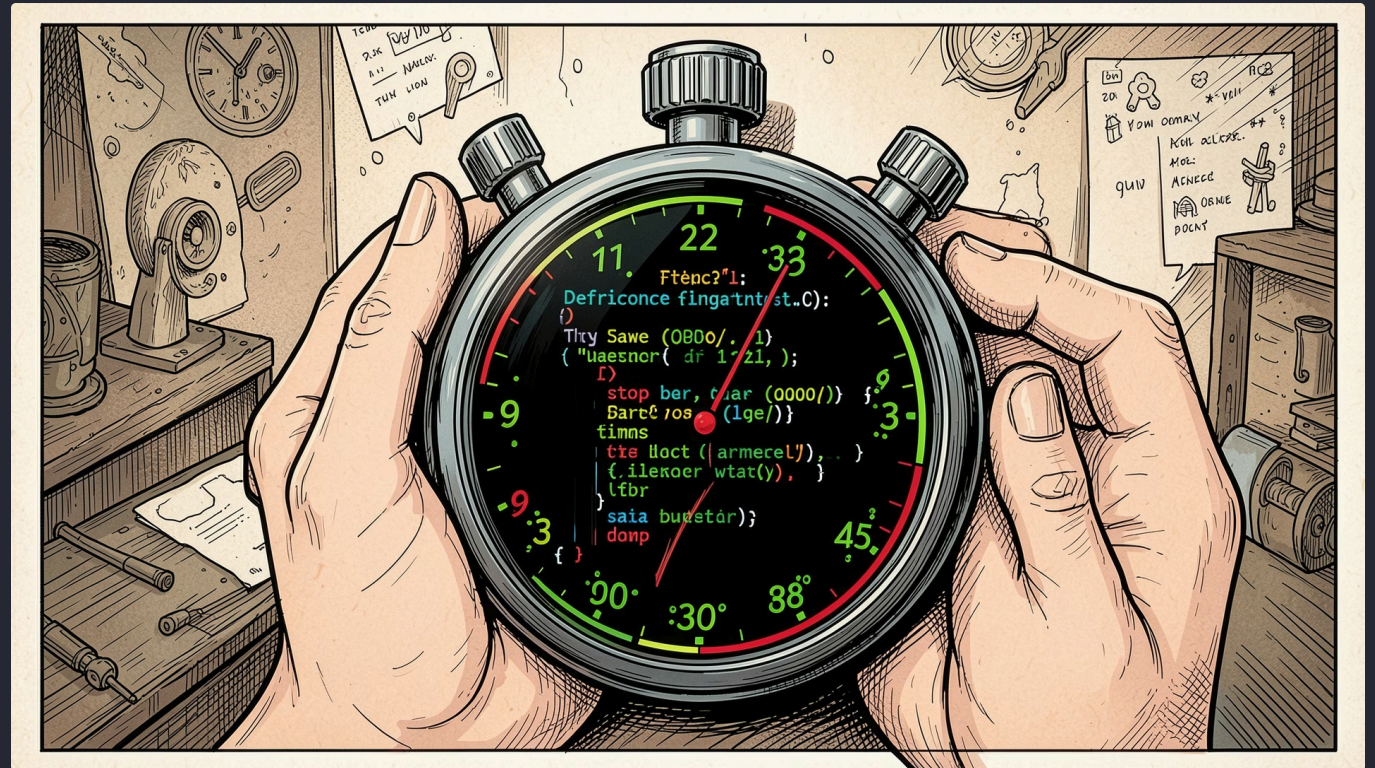
$O(n + m)$

Limites do problema

Até 100.000 vértices · 200.000 arestas

⚠️ Time Limit Exceeded

Solução: **FastIO** para leitura rápida



Resultado Obtido

✓ Aceito no CSES

Todos os casos de teste
aprovados

🎯 Dijkstra

Ideal para pesos não negativos

⚡ Eficiência

Lista de adjacência + PriorityQueue

CSES Problem Set

Shortest Routes I

[TASK](#) | [SUBMIT](#) | [RESULTS](#) | [ANALYSIS](#) | [STATISTICS](#) | [TESTS](#) | [QUEUE](#)

Submission details

Task:	Shortest Routes I
Sender:	lkzmagalhaes
Submission time:	2026-05-31 19:40:21 +0300
Language:	Java
Status:	READY
Result:	ACCEPTED

Test results ▲

test	verdict	time	
#1	ACCEPTED	0.07 s	»»
#2	ACCEPTED	0.08 s	»»
#3	ACCEPTED	0.08 s	»»
#4	ACCEPTED	0.08 s	»»
#5	ACCEPTED	0.07 s	»»
#6	ACCEPTED	0.45 s	»»
#7	ACCEPTED	0.48 s	»»
#8	ACCEPTED	0.49 s	»»
#9	ACCEPTED	0.47 s	»»
#10	ACCEPTED	0.53 s	»»
#11	ACCEPTED	0.34 s	»»
#12	ACCEPTED	0.39 s	»»
#13	ACCEPTED	0.07 s	»»
#14	ACCEPTED	0.38 s	»»
#15	ACCEPTED	0.37 s	»»
#16	ACCEPTED	0.35 s	»»
#17	ACCEPTED	0.32 s	»»
#18	ACCEPTED	0.51 s	»»
#19	ACCEPTED	0.47 s	»»
#20	ACCEPTED	0.46 s	»»
#21	ACCEPTED	0.45 s	»»
#22	ACCEPTED	0.53 s	»»
#23	ACCEPTED	0.41 s	»»
#24	ACCEPTED	0.43 s	»»

Made with GAMMA

Caso Especial Considerado

Como as distâncias podem crescer muito durante a execução do Dijkstra, utilizamos `long` para evitar **overflow** e garantir a correte da solução.



Pesos Elevados

Pesos de aresta podem atingir 10^9 , e um caminho pode acumular vários desses valores.

1

Uso de `long`

Adotamos o tipo `long` para armazenar as distâncias, garantindo capacidade suficiente.



Potencial Overflow

O valor total da distância acumulada pode exceder o limite máximo de um tipo `int` padrão.



Correte Assegurada

Isso assegura resultados precisos e corretos, mesmo para os maiores casos de teste.